

Fake Gacha Backend Project

Created by: Daniel Hor Jung Wey

This document explains and shows the process of creating a fake mobile game backend designed specifically to demonstrate real-world cloud (focusing on using AWS) and application security skills. The goal is not to build a game, but to build and defend a realistic API-driven backend that attackers would target.

My plan is to build a server-side backend that behaves like the backend of a gacha style mobile game, such as:

- User authentication and session management
- Player state management (example gems, stamina, inventory)
- Game actions (gacha pulls, daily rewards)
- Monitoring, detection, and automated response to abuse

I am proving that I understand how attackers actually compromise modern systems. I demonstrate that:

- I understand API abuse and business logic attacks
- Correct handling of authentication vs authorization
- Defense against IDOR (Insecure Direct Object Reference), token replay, and automation
- Use of AWS-native security controls rather than ad-hoc logic

プロジェクト概要

本プロジェクトは、ガチャ系モバイルゲームのサーバーサイドバックエンドをAWSで構築した実践記録です。ゲーム開発が目的ではなく、実際の攻撃者が標的にするようなAPIドリブンなバックエンドを設計・構築し、それを防御することを目的としています。

クラウドおよびアプリケーションセキュリティの実践的なスキルを証明するために作成しました。

使用したAWSサービス

サービス	役割
Amazon Cognito	ユーザー認証・JWTトークンの発行
AWS Lambda	サーバーレスバックエンドロジック(Python)

Amazon API Gateway (REST API)	HTTPエンドポイントの管理・Cognito認可
Amazon DynamoDB	プレイヤーデータの永続化
AWS IAM	最小権限の原則に基づくサービス間アクセス権限の管理
AWS WAF	レートベースルールによるAPIアビュース防御
Amazon CloudWatch	WAFブロックイベントの監視・SNSアラート通知

実装した機能

- ユーザー認証 — CognitoによるJWTベースの認証フロー (OAuth 2.0)
 - プレイヤープロフィール取得 — GET /profile (DynamoDBからプレイヤーデータを返す)
 - ガチャ機能 — POST /gacha (ジェムの消費・アイテムのインベントリへの追加)
-

セキュリティの観点

AWSネイティブのセキュリティ機能を活用し、以下の点を意識して設計・実証しました。

- IDOR (Insecure Direct Object Reference) への対応 — userIDをURLパラメータから取得せず、署名済みJWTトークンから取得することで、他ユーザーのデータへの不正アクセスを防止。第2テストユーザーによる攻撃シナリオで実証済み
- APIレート制限 — AWS WAFのレートベースルールにより、5分間に100リクエストを超えたIPをブロック。Postman Collection Runnerで150回のリクエストを送信し、動作を確認済み
- トークンリプレイ攻撃への対応 — CognitoのJWTトークンは1時間で失効、HTTPS通信のみ許可することで盗聴リスクを低減
- 最小権限の原則 — getUserProfile LambdaにはDynamoDB読み取り権限のみ、gachaPull LambdaにはDynamoDB読み書き権限を付与し、機能ごとに必要最小限の権限を設定
- 監視・アラート — CloudWatchアラームがWAFのブロックイベントを検知し、SNS経由でメール通知を送信

1. Creating simple user login backend

1.1 Using Amazon Cognito

Defined the application, selected mobile app and used email for sign-in identifiers (only used email required for sign-up). Set return URL to <http://localhost> for temporary testing

Configure options

You must make a few initial choices about the user pool that supports your application. To change these settings later, you must create a new user pool.

Options for sign-in identifiers | [Info](#)

Choose sign-in attributes. Usernames can be an email address, phone number, or a user-selected username. When you select only email and phone, users must select either email or phone as their username type. When username is an option, users can sign in with any options you select if they have provided a value for that option.

- ☒ Email
- ☐ Phone number
- ☐ Username

[Want to set up social, SAML, or OIDC sign-in?](#)

Self-registration | [Info](#)

If you activate user sign-up in your user pool, anyone on the internet can sign up for an account and sign in to your apps. Don't enable self-registration in your user pool until you want to open your app to public sign-up. [Learn more](#)

- ☒ Enable self-registration
Display a "Sign up" link on the sign-in page in the hosted UI, and allow the use of public APIs to create new user accounts. When this feature is not enabled, federation and administrative API operations create user profiles.

Required attributes for sign-up | [Info](#)

Choose any attributes that you want to require users to provide. With username alone, you must set email address or phone number as a required attribute.

Select attributes

email
User's preferred email address.

 Options for sign-in identifiers and required attributes can't be changed after the app has been created.

Add a return URL - optional

Choose a return URL. Cognito redirects to this URL after successful sign-in with the managed login pages on your user pool domain. Your application can then process the resulting tokens.

Return URL | [Info](#)

<http://localhost>

The length of the return URL must be between 1 and 1024 characters. Valid characters are letters, marks, numbers, symbols, and punctuations. Amazon Cognito requires HTTPS over HTTP except for <http://localhost> for testing purposes only. App return URLs such as myapp://example are also supported. Must not contain a fragment.

Once created, I went to User Management and created a test user, without making an invitation message just for testing. Secondly, I went to App clients -> Login pages and managed login

pages configuration. I changed the OAuth grant type from Authorization Code Grant to Implicit grant for temporary testing. Will switch back to OAuth later as JWT token is not exposed in URL

Difference between Authorization Code Grant and Implicit Grant

Authorization Code

The authorization code grant is an OAuth 2.0 flow (a framework that lets an application access a user's resources without ever seeing their password. Basically OAuth makes it so that apps get limited access tokens that can be short-lived, scoped and revoked) where the client first receives a short-lived authorization code, which is then exchanged server-to-server for tokens such as the id_token and access_token. The browser gets the code, and the backend gets the tokens. This exists so that it prevents token leakage, and keeps the token away from the browser.

Implicit Grant

The identity provider directly returns tokens (like id_token) to the client via the browser URL which does not require a backend exchange. This means that the token is exposed in the browser. This is weaker due to it can be leaked via browser history, logs etc

1.2 Amazon Lambda - Simple response

Created a function called getUserProfile, runtime Python 3.X. Created a simple function to prove that if someone calls the backend, the backend can respond. Function name is getUserProfile, added code

```
def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "body": "You reached the backend"
    }
```

This code makes Lambda return a basic response.

1.3 Create HTTP API Gateway

Created a HTTP API Gateway that gives my Lambda function an HTTP endpoint.

Selected my Lambda function so that the API knows which backend should I send my request to. Next, I selected GET method, and resource path /profile. This means that if someone sends a GET request to /profile, I call the Lambda function.

I then went to Deploy -> Stages, and used the Invoke URL to test whether my Lambda function is being called. I added /profile at the end of the URL (route).

Simple definition of API (Application Programming Interface)

API name

An HTTP API must have a name. The name is a non-unique value you use to identify and organize your APIs. To programmatically refer to this API, use the API ID that API Gateway generates for you.

IP address type [Info](#)

Select the type of IP addresses that can invoke the default endpoint for your API. You don't need to redeploy your API for the update to take effect.

- ☒ IPv4
Includes only IPv4 addresses.
- ☐ Dualstack
Includes IPv4 and IPv6 addresses.

Integrations (1) [Info](#)

Specify the backend services that your API will communicate with. These are called integrations. For a Lambda integration, API Gateway invokes the Lambda function and responds with the response from the function. For an HTTP integration, API Gateway sends the request to the URL that you specify and returns the response from the URL.

[Remove](#)

AWS Region

Lambda function

Version

[Learn more](#)[Add integration](#)

It is a way for one system to talk to another system using defined rules. In this case, it checks what path (/profile) and what method (GET) and sends it to my lambda function.

1.4 Creating a JWT Authorizer

Now the goal is to make it so that only requests with a valid JWT (id_token) can reach Lambda because currently anyone can access it.

Identity source: `$request.header.Authorization` means look for the token in the authorization header. This step tells the API Gateway that before calling Lambda check whether the request has a valid JWT from AWS Cognito.

Create authorizer

Authorizer type [Info](#)

☒ JWT
Use JSON Web Tokens (JWTs) to control access to your API.

☐ Lambda
Use a Lambda function to control access to your API.

Authorizer settings

Name

Enter a name for the authorizer.

Identity source

Enter a selection expression that defines the source of the token.

Issuer URL

Enter the issuer URL of your identity provider, commonly found in the issuer field of the Authorization Server's well-known metadata endpoint.

Audience

Enter client IDs that are registered with identity providers or any arbitrary string in the JWT audience claim that the authorizer must verify.

[Remove](#)[Add audience](#)

Secondly, we need the Issuer URL and Audience, so that the API knows to only trust tokens that come from my Cognito User Pool.

To be able to get the Issuer URL, I went to Cognito, opened the User Pool and looked for the User pool ID. Following this format:

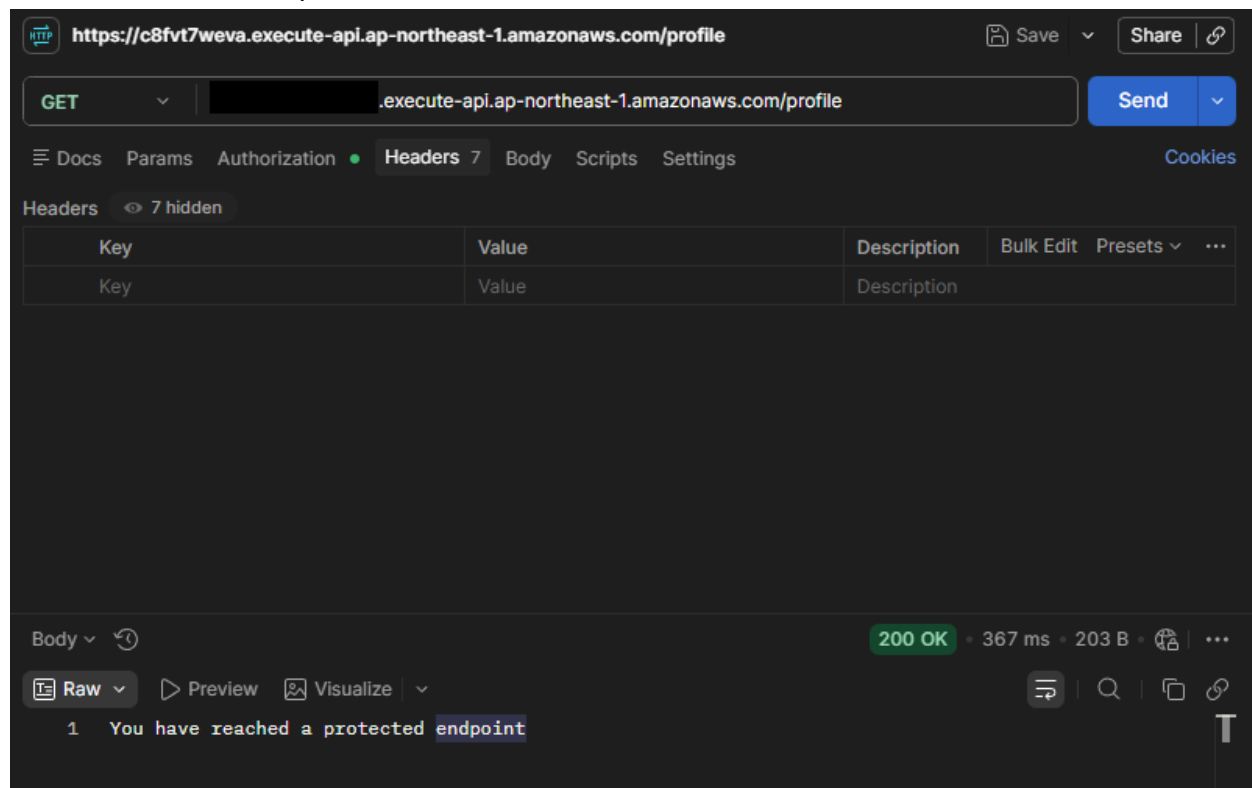
<https://cognito-idp.<region>.amazonaws.com/<userPoolId>>

Next for the Audience (Client ID) which is my fake mobile game backend, I went to App clients and copied my Client ID. This ensures that this token was meant for this application.

By doing this the API Gateway will now verify whether signature is valid, issuer matches, audience matches and the token is not expired.

Next step is to attach the authorization to /profile by going to Routes and clicking on the appropriate route.

After attaching, to test it out, I used Postman to test whether someone with a valid JWT token is able to access the endpoint.



To do this, go to Authorization, Auth Type (Bearer Token) and copy the JWT token given by the URL (after the #id_token and ends before &access_token). The API Gateway receives the request and sees there's an authorizer attached to the /profile route.

The JWT Authorizer checks 4 things:

- **Signature** — was this token actually signed by Cognito? (Can't be faked)
- **Issuer** — does it come from YOUR Cognito user pool?
- **Audience** — was it made for YOUR specific app (Client ID)?

- **Expiry** — is it still valid, or has the 1 hour passed?

2. Creating Player State Storage and Gacha

2.1 Setting up DynamoDB

My next step is to create a database to store every players unique user ID using the sub field in the decoded JWT. Went to DynamoDB -> Create Table and named it PlayerProfile. Partition key is userId (type is string).

When a player logs in, Cognito puts their sub into the token.

The screenshot shows a JWT token decoder interface. On the left, under 'Encoded Token', there is a long string of base64-encoded characters. On the right, under 'Decoded Header', the JSON structure is shown with fields 'kid' and 'alg'. Below that, under 'Decoded Payload', the full JSON structure is displayed, including fields like 'at_hash', 'sub', 'iss', 'cognito:username', 'aud', 'event_id', 'token_use', 'auth_time', 'exp', 'iat', 'jti', and 'email'.

```

Encoded Token
[ ] Enable auto-focus
Decoded Header
JSON Claims Breakdown
{
  "kid": "YX5Ng82WAFxYIikRQm9VSLqseoBA6cspsrca+wyv550=",
  "alg": "RS256"
}

Decoded Payload
JSON Claims Breakdown
{
  "at_hash": "M_eGnWbeDQC90BOVEjJkma",
  "sub": "[REDACTED]",
  "iss": "https://cognito-idp.ap-northeast-1.amazonaws.com/[REDACTED]",
  "cognito:username": "b7440a48-80b1-708b-f841-b98adb54d6bb",
  "aud": "[REDACTED]",
  "event_id": "6188df61-3460-40b1-ab2c-3ecf95f40f82",
  "token_use": "id",
  "auth_time": 1780802844,
  "exp": 1780806444,
  "iat": 1780802844,
  "jti": "8b3ea330-1c26-4ef1-b0f5-b9cdd6c150f1",
  "email": "[REDACTED]"
}

```

After that., created one user using the sub from the JWT token, and created 3 sub-fields “gems”, “inventory” and “stamina”

Table: PlayerProfile - Items returned (1)

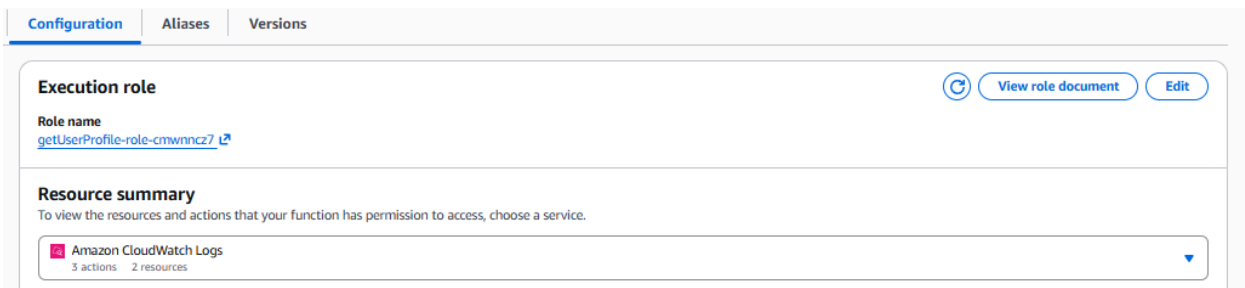
Scan started on June 07, 2026, 12:32:30

	userId (String)	gems	inventory	stanima
☐	b7440a48-80b1-708...	100	[]	50

Important: inventory must be set as a List type, not String. list_append() in DynamoDB requires an actual List to append to."

2.2 Giving Lambda permission to read DynamoDB

The next step is to give Lambda permissions (IAM) to be able to read from the database I have created. To do that, I went to Lambda, clicked on getUserProfile -> Configurations -> Permissions and clicked the Execution role link.



Once clicked, it leads to IAM, where I attached the AmazonDynamoDBReadOnlyAccess policy to the Lambda function. Read only to follow principle of least privilege. Next step is to update the lambda code to pull out the "sub" out of the event.

```
lambda_function.py
1  import json
2  import boto3
3  from decimal import Decimal
4
5  dynamodb = boto3.resource('dynamodb')
6  table = dynamodb.Table('PlayerProfile')
7
8  class DecimalEncoder(json.JSONEncoder):
9      def default(self, obj):
10         if isinstance(obj, Decimal):
11             return int(obj)
12         return super().default(obj)
13
14  def lambda_handler(event, context):
15      user_id = event['requestContext']['authorizer']['jwt']['claims']['sub']
16
17      response = table.get_item(Key={'userId': user_id})
18      item = response.get('Item')
19
20      if not item:
21          return {
22              'statusCode': 404,
23              'body': json.dumps({'message': 'Player not found'})
24          }
25
26      return {
27          'statusCode': 200,
28          'body': json.dumps(item, cls=DecimalEncoder)
29      }
```

Note: when `table.get_item` cannot find anything, it returns `None`, triggering the 404.

`DecimalEncoder` is required due to DynamoDB using decimal for all numbers. Python `json.dumps()` does not know how to handle decimals.

`table.get_item` looks up the player row in DynamoDB using the `user_id`. Returns the player data if found, 404 if not. Below is a visualizer on what the API Gateway sends to the Lambda when the event is called.

```
{
  "version": "2.0",
  "routeKey": "GET /profile",
  "requestContext": {
    "authorizer": {
      "jwt": {
        "claims": {
          "sub": [REDACTED] ← Lambda uses this
          "email": [REDACTED]
          "aud": "[REDACTED]"
          "token_use": "id",
          "exp": 1780317489
        }
      }
    }
  },
  "http": { "method": "GET", "path": "/profile" } route info
},
  "headers": { "authorization": "Bearer eyJraWQiOiJZbD..." } your JWT token
}
```

The code maps directly to the tree, each word in the line is one level deeper into the JSON.

2.3 Lambda function for gacha pull

Now I am going to create a Lambda function to simulate the gacha pulls. I do this by creating a new function called `gachaPull` and adding the said code:

```
import json
import boto3 #boto3 is the official AWS SDK for python
```

```

import random
from decimal import Decimal

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('PlayerProfile')

class DecimalEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, Decimal):
            return int(obj)
        return super().default(obj)

GACHA_ITEMS = ['sword', 'shield', 'potion', 'bow', 'staff']
GACHA_COST = 10

def lambda_handler(event, context):
    user_id = event['requestContext']['authorizer']['claims']['sub']

    response = table.get_item(Key={'userId': user_id})
    item = response.get('Item')

    if not item:
        return {
            'statusCode': 404,
            'body': json.dumps({'message': 'Player not found'})
        }

    current_gems = int(item['gems'])
    if current_gems < GACHA_COST:
        return {
            'statusCode': 400,
            'body': json.dumps({'message': 'Not enough gems'})
        }

    pulled_item = random.choice(GACHA_ITEMS)

    table.update_item(
        Key={'userId': user_id},
        UpdateExpression='SET gems = gems - :cost, inventory = list_append(inventory, :item)',
        ExpressionAttributeValues={
            ':cost': Decimal(GACHA_COST),
            ':item': [pulled_item]
        }
    )

    return {
        'statusCode': 200,
        'body': json.dumps({
            'message': 'Gacha pull successful',
            'item_received': pulled_item,
            'gems_remaining': current_gems - GACHA_COST
        }, cls=DecimalEncoder)
    }

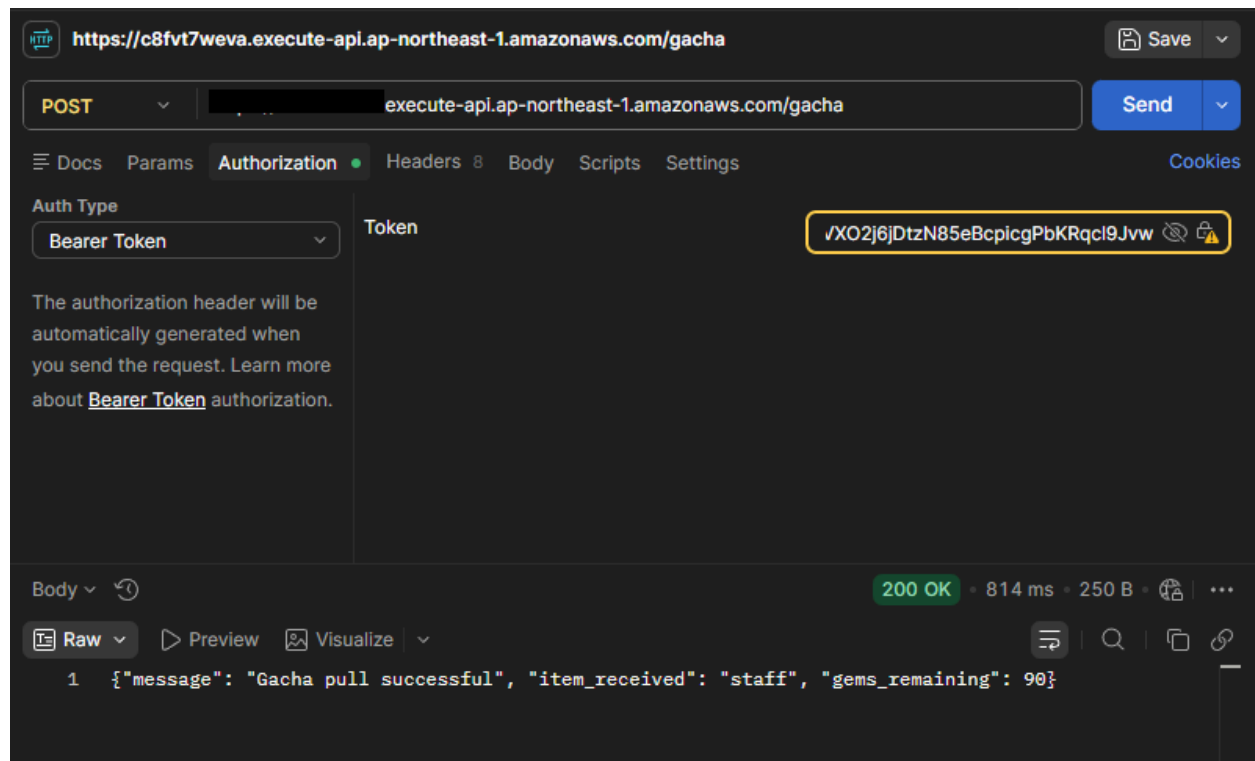
```

Since this new Lambda function requires permission to write to DynamoDB, we create a new configuration and attach the “AmazonDynamoDBFullAccess” policy.

After this step, we need to add the Gacha route to the API Gateway. In the existing getUserProfile API, we create a new route POST /gacha, and then attach the Lambda integration. Same for the Authorization, we attach the same cognito-auth authorizer.

NOTE: Due to migrating to REST API, changed from getUserProfileAPI to the REST API (GachaGameAPI). In there two resources were created, GET /profile and POST /gacha

After that, I am able to test it out in Postman. Invoke URL + /gacha and POST instead of GET.



2.4 Migrating to REST API Gateway

After the initial HTTP API setup, I decided to migrate to a REST API to enable AWS WAF (Web Application Firewall) attachment for rate limiting and abuse prevention. HTTP API does not

support WAF, it is a lightweight version of API Gateway optimised for speed and cost, but lacks advanced security features.

Key differences between HTTP API and REST API:

	HTTP API	REST API
WAF support	✗	✓
Auto-deploy	✓	✗ Manual deploy required
Authorizer type	JWT (manual issuer + audience config)	Cognito native (select user pool directly)
Event structure	<code>event['requestContext']['authorizer']['jwt']['claims']['sub']</code>	<code>event['requestContext']['authorizer']['claims']['sub']</code>
Cost	Cheaper	Slightly more expensive

The migration required rebuilding the API from scratch. Creating new resources, methods, and a new Cognito authorizer. Created a new REST API called “GachaGameAPI”, and added two resources, /gacha (POST) and /profile (GET). When creating the method, Lambda proxy integration is enabled. Without Lambda proxy integration enabled, API Gateway strips the entire event object and Lambda receives an empty {}. This causes a KeyError when trying to read requestContext, since the key doesn't exist. This is confirmed by temporarily returning the raw event from Lambda and seeing "body": "{}" in Postman.

Edit integration request

Method details

Integration type

☒ **Lambda function**
Integrate your API with a Lambda function.


☐ **HTTP**
Integrate with an existing HTTP endpoint.


☐ **Mock**
Generate a response based on API Gateway mappings and transformations.


☐ **AWS service**
Integrate with an AWS Service.


☐ **VPC link**
Integrate with a resource that isn't accessible over the public internet.


☒ **Lambda proxy integration**
Send the request to your Lambda function as a structured event.

Response transfer mode [Info](#)

- ☒ **Buffered**
Wait to receive the complete response before beginning transmission.
- ☐ **Stream**
Send portions of the response without waiting for the complete response.

Lambda function

Provide the Lambda function name or alias. You can also provide an ARN from another account.

ap-northeast-1

With proxy integration enabled, API Gateway passes the entire raw request to Lambda untouched instead of stripping most of it.

```
python

{
  "requestContext": {
    "authorizer": {
      "claims": {
        "sub": "b7440a48...",
        "email": [REDACTED]
      }
    }
  },
  "headers": { ... },
  "body": { ... }
}
```

The Lambda functions also needed updating because the event structure is different between the two API types. In REST API, the JWT layer is removed and claims are accessed directly under “authorizer”.

In REST API, Cognito has its own dedicated authorizer type. In API Gateway, went to Authorizers and created a new authorizer called “cognito-auth” and selected my created user pool.

Create authorizer [Info](#)

Authorizer details
Authorizer name
cognito-auth

Authorizer type [Info](#)
Choose to authorize your API calls using one of your Lambda functions or a Cognito User Pool.
☐ Lambda
☒ Cognito

Cognito user pool
Select the Cognito user pool that will authenticate requests to your API.
ap-northeast-1 User pool - [REDACTED]

Token source
Enter the header that contains the authorization token. The prefix method.request.header. will be automatically prepended.
method.request.header. Authorization

Token validation - optional
Enter a regular expression to validate the aud field of the identity token.

[Cancel](#) [Create authorizer](#)

After I have created the authorizer, went back to the resources and under Method Request, added the cognito-auth.

Method Request — the front door

- Controls what comes IN to API Gateway
- Things like: does this request need authorization? What headers are required?
- This is where I attached cognito-auth

Integration Request — the back door

- This is what faces Lambda
- Controls how API Gateway talks to your backend
- Things like: which Lambda function to call? How to pass the data?
- This is where Lambda proxy integration lives

3. Setting Up WAF

3.1 What am I defending against?

The gacha endpoint POST /gacha is the most obvious attack target in this backend. A real attacker wouldn't try to crash the server, they would write a script to automate pulls, farming rare items faster than any human could while bypassing the intended game economy. This is called API abuse or business logic abuse: exploiting the game's own mechanics through automation rather than trying to take the server down.

This is more dangerous than a plain DoS attack because:

- The server stays up and nobody notices anything is wrong
- The attacker is quietly gaining game advantage
- It looks like normal traffic on the surface

The defense is rate limiting. Blocking IPs that exceed a request threshold within a time window.

3.2 Why WAF and not just API Gateway throttling?

API Gateway has built-in throttling, but AWS WAF provides more control and visibility:

- WAF blocks at the network edge before the request even reaches API Gateway
- WAF provides sampled request logs showing exactly what was blocked and why

- WAF rules can be expanded later, for example: IP blocklists, geo-blocking, SQL injection protection

This is also why the migration from HTTP API to REST API was necessary. HTTP API does not support WAF attachment.

3.3 Creating the Web ACL

Created a Web ACL called GachaGameWAF in WAF & Shield, attached to GachaGameAPI - dev stage. Selected "You build it" protection pack to avoid unnecessary costs from managed rule groups.

Added one custom Rate-based rule called GachaRateLimit:

Setting	Value	Reasoning
Rate limit	100 requests	A real player maxes out at 10-20 pulls per minute. 100 per 5 minutes is generous for humans but immediately catches bots
Evaluation window	5 minutes	Standard window. Short enough to catch bursts, long enough to not false-positive real players
Aggregation	Per IP address	Each IP gets its own counter
Action	Block	Hard block the IP, not just log it

3.4 Testing the rate limit

To verify the rule works, I used Postman Collection Runner to simulate automated API abuse:

- 150 iterations at 0ms delay to simulating a bot spamming requests
- First ~100 requests returned 200 OK. Pulls succeeding normally
- After crossing the threshold ->403 Forbidden -> WAF blocking the IP

This confirms the rate limit is working correctly.

My Collection - Run results

Ran today at 10:11:38 PM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	150	15s 956ms	0	0	17 ms

All 0 Passed 0 Failed 0 Skipped 0 Errors 0 Console log

Iteration 1

POST [redacted].execute-api.ap-northeast-1.amazonaws.com/dev/gacha

https://[redacted].execute-api.ap-northeast-1.amazonaws.com/dev/gacha 403 • 51 ms • 280 B

No tests found

Iteration 2

POST [redacted].execute-api.ap-northeast-1.amazonaws.com/dev/gacha

https://[redacted].execute-api.ap-northeast-1.amazonaws.com/dev/gacha 403 • 18 ms • 280 B

No tests found

Iteration 3

POST [redacted].execute-api.ap-northeast-1.amazonaws.com/dev/gacha

https://[redacted].execute-api.ap-northeast-1.amazonaws.com/dev/gacha 403 • 14 ms • 280 B

No tests found

Iteration 4

POST [redacted].execute-api.ap-northeast-1.amazonaws.com/dev/gacha

https://[redacted].execute-api.ap-northeast-1.amazonaws.com/dev/gacha 403 • 14 ms • 280 B

No tests found

WAF counts all requests from an IP regardless of whether they are authorized or not. Failed requests with invalid tokens still increment the rate limit counter. This means the effective threshold a legitimate user hits may be lower if they previously sent invalid requests from the same IP.

4. Demonstrating protection against IDOR

4.1 What is IDOR

IDOR (Insecure Direct Object Reference) is when an attacker can access someone else's data by simply changing an ID in the request. A classic example in a vulnerable API:

GET /profile?userId=victim123 → returns the victim's data

If the backend trusts the userId from the URL without verifying ownership, any authenticated user can read anyone else's data just by guessing IDs.

4.2 Why this backend is IDOR-safe

In a vulnerable design, the backend would take the `userId` directly from the request:

```
user_id = event['queryStringParameters']['userId']
```

This backend never does that. Instead, the `userId` is always extracted from the verified JWT token:

```
user_id = event['requestContext']['authorizer']['claims']['sub']
```

The player cannot lie about their `userId` because the token is signed by Cognito. Any modification to the token (for example `/profile?userId= (sub)`) invalidates the signature and the request is rejected before it even reaches Lambda.

4.3 Demonstrating the protection

To prove this, a second Cognito test user was created to act as the attacker. The attacker has their own DynamoDB entry with 999 gems, deliberately different from the victim's 100 gems to make any data leak immediately obvious.

The attack attempt:

GET `/profile?userId=(user1 sub)`

The attacker passes the victim's `userId` in the URL while using their own valid JWT token.

The result:

The screenshot shows a web browser's developer tools interface. The 'Authorization' tab is selected, showing 'Auth Type' as 'Bearer Token' and a 'Token' field with a masked value. Below this, a message states: 'The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.'

The 'Body' tab is selected, showing the response body in JSON format:

```
1 {
2   "inventory": [],
3   "gems": 999,
4   "userId": "(user1 sub)",
5   "stamina": 50
6 }
```

The API returned the attacker's own data — not the victim's. The `?userId=` parameter in the URL was completely ignored. The backend always reads identity from the JWT, not from anything the client can control.

This confirms the backend is protected against IDOR by design — not by adding extra validation logic, but by never trusting client-supplied IDs in the first place.

5. CloudWatch Monitoring & Alerting

5.1 CloudWatch and setting it up

WAF blocks attacks without monitoring, there's no way to know an attack is happening in real time. CloudWatch fills this gap by tracking metrics and triggering alerts when thresholds are crossed.

Created a CloudWatch metric alarm called `GachaAlerts` with the following configuration:

Setting	Value	Reasoning
Metric	BlockedRequests	Tracks requests blocked by WAF
Namespace	AWS/WAFV2	WAF-specific metrics
WebACL	GachaGameWAF	Our specific WAF rule set
Rule	GachaRateLimit	The rate limiting rule specifically
Statistic	Sum	Total blocked requests, not average

Threshold Greater than 5 Triggers on meaningful block activity

Evaluation period 5 minutes Matches WAF rate limit window

Important: Statistics must be set to Sum, not Average. Average divides blocked requests across time periods, producing small decimal values (e.g. 1.1, 3.05) that never cross the threshold. Sum counts all blocked requests in the window, correctly spiking above the threshold during an attack.

5.2 SNS Email Notification

Connected the alarm to an SNS (Simple Notification Service) topic called GachaAlerts with an email subscription. When the alarm triggers, SNS sends an immediate email notification.

What is SNS? SNS is AWS's messaging service. It acts as a middleman between CloudWatch and the notification endpoint. CloudWatch tells SNS "the alarm triggered", and SNS forwards the alert to all subscribers (in this case, email).

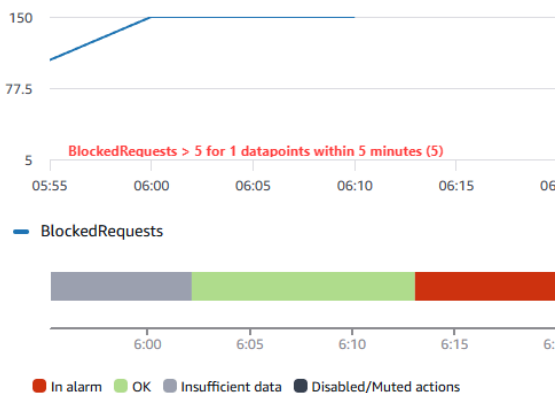
To verify the full monitoring pipeline works:

1. Ran Postman Collection Runner — 150 iterations at 0ms delay
2. WAF blocked requests after crossing the rate limit threshold
3. CloudWatch detected BlockedRequests sum exceeding 5
4. SNS sent an email notification confirming the alarm triggered

Graph

BlockedRequests

BlockedRequests > 5 for 1 datapoints within 5 minutes



This confirms the complete defense and monitoring loop:

Bot spams API → WAF blocks requests → CloudWatch detects spike → SNS sends email alert

WAF uses a **sliding 5 minute window**, not a fixed reset. Running 150 requests twice in quick succession caused blocking at iteration 43 of the second run because the window still contained requests from the first run. This makes the rate limit more effective against sustained attacks than a simple fixed window reset would be.

Monitored Metric:

- MetricNamespace:	AWS/WAFV2
- MetricName:	BlockedRequests
- Dimensions:	[WebACL = GachaGameWAF] [Region = ap-northeast-1] [Rule = GachaRateLimit]
- Period:	300 seconds
- Statistic:	Sum
- Unit:	not specified
- TreatMissingData:	missing